

Is the false-positive cascade specific to multidimensional misfit?

Random responding vs multidimensionality at matched misfit magnitude

Magnus Johansson

2026-08-01

Table of contents

Scope	1
Design	1
Calibration	1
Run	3
The cascade: false positives on fitting items	5
Direction of the false flags	5
Detection of the genuinely misfitting items	6
Does FWER keep its advantage under the other mechanism?	6
References	8

Scope

Every result in the width/iterations series uses **one** misfit mechanism: items loading on a correlated second latent dimension. Most of the paper's claims cannot depend on that choice —

- the Sidak relation $1 - w^k$, the B-convergence results and Type I error are properties of the **null** distribution and the decision rule; with no misfitting items there is no mechanism;
- FWER's advantage over the width rule comes from the Westfall-Young step-down exploiting dependence among item statistics rather than from the form of the alternative;
- instability tracks how many items sit near the decision boundary, which is close to definitional.

One claim is genuinely at risk: the false-positive cascade. Under multidimensionality the misfitting items form a *correlated cluster*, so the total score is contaminated systematically. That is the mechanism behind the large-sample false-positive rates documented in the earlier studies (54% for infit at $n = 2000$ dichotomous; 92-100% for polytomous at $n = 1000$), and behind the correction those results imply for how Johansson (2025) is read.

Random responding also contaminates the score, so a cascade is expected — the question is **magnitude, not presence**. If it is much weaker, the headline false-positive results are a property of *multidimensional* misfit specifically, and the paper must scope them accordingly.

Other mechanisms (local dependence, genuine discrimination variation) both produce **overfit**, which item fit alone cannot attribute — see Tuerlinckx & De Boeck (2001). They belong with Q3 in a separate paper and are not studied here.

Design

Built as a direct comparator to the existing $\rho = 0.85$ cells in `infit_width_iterations_poly_weak.qmd`: same arm (phq9, 9 items with CML parameters from the packaged real responses, latent SD 1.39), same misfitting items (**q5** at -0.15 and **q8** at $+1.40$), same k , n , B , widths and replication count. Only the mechanism changes.

Random responding: a proportion p of responses on each target item is replaced by a uniform draw over the response categories (0-3). Same approach as `dev/infit_dgp_fwer.R`.

Calibration

$p = 0.15$, chosen so the target item's expected infit matches the multidimensional comparator at $k = 1$:

k	Multidimensionality ($\rho = 0.85$)	Random responding ($p = 0.15$)
1	1.251	1.260
2	1.164	1.263

Calibration is anchored at **$k = 1$ only**, deliberately. Under multidimensionality the two misfitting items are correlated with each other and **mask** one another, so their infit falls at $k = 2$. Independent noise items do not mask, so infit is unchanged. Forcing a match at $k = 2$ as well would hide that difference; leaving p fixed makes it an outcome.

Expected infit is a property of the data-generating process, so it is essentially invariant to sample size (verified earlier: 2.03 vs 2.06 at $n = 250$ vs 1000 for a fixed condition). Targeting affects detectability strongly but magnitude only slightly. Infit is item-set dependent — it is a relative index and the Rasch constraint pins the average near 1 — which is why this study stays within a single arm.

```
suppressMessages({
  pkgload::load_all("../", quiet = TRUE)
  library(ggdist)
  library(parallel)
  library(dplyr)
  library(tidyr)
  library(ggplot2)
})

NOISE_P <- 0.15
WIDTHS <- c(0.90, 0.95, 0.99, 0.995, 0.999)
NS <- c(250, 1000)
BS <- c(100, 400, 2000)
KS <- c(1L, 2L)
R_REPS <- 120
NCORES <- 10
ALPHA <- 0.05
CHUNK <- 60
RES <- "infit_misfit_mechanism.rds"
COMPARATOR <- "infit_width_iterations_poly_weak.rds"
```

```
run_resumable <- function(res_file, grid, keys, fn,
                          ncores = NCORES, chunk_size = CHUNK,
                          extra = list()) {
  key_of <- function(g) {
    if (nrow(g) == 0L) return(character(0))
    do.call(paste, c(unname(as.list(g[keys])), sep = "|"))
  }
  acc <- if (file.exists(res_file)) readRDS(res_file) else
    c(list(res = list(), grid = grid[0, , drop = FALSE], elapsed_min = 0), extra)
  todo <- which(!(key_of(grid) %in% key_of(acc$grid)))
  if (length(todo) == 0L) {
    message("cached: ", nrow(acc$grid), " cells, nothing to run"); return(acc)
  }
  message("running ", length(todo), " new cells (", nrow(acc$grid), " cached)")
  for (s in seq(1, length(todo), by = chunk_size)) {
    idx <- todo[s:min(s + chunk_size - 1L, length(todo))]
    t0 <- Sys.time()
    out <- mclapply(idx, function(i) fn(grid[i, , drop = FALSE]), mc.cores = ncores)
    acc$res <- c(acc$res, out)
    acc$grid <- rbind(acc$grid, grid[idx, , drop = FALSE])
    acc$elapsed_min <- acc$elapsed_min + as.numeric(Sys.time() - t0, "mins")
    acc <- utils::modifyList(acc, extra)
    saveRDS(acc, res_file)
    message(" checkpoint ", nrow(acc$grid), " / ", nrow(grid),
            " (", round(acc$elapsed_min, 1), " min)")
  }
  acc
}
```

```
load("../data/phq9.rda")
PH_OBS <- as.matrix(phq9[, paste0("q", 1:9)])
PH_OBS <- PH_OBS[complete.cases(PH_OBS), ]
PH_THR <- .fit_cml_thresholds(PH_OBS)
PH_LOCS <- vapply(PH_THR, mean, numeric(1))
.g <- seq(-6, 6, length.out = 81)
PH_SD <- .estimate_prior_sd(
  .grid_loglik(PH_OBS, .logp_tables(PH_THR, .g), .g), .g, 0)
```

```

PH_MIS <- c(5L, 8L)
MAXCAT <- 3L

sim_noise <- function(n, k, seed) {
  set.seed(seed)
  th <- rnorm(n, 0, PH_SD)
  d <- sim_partial_score(PH_THR, th)
  for (it in PH_MIS[seq_len(k)]) {
    fl <- runif(n) < NOISE_P
    if (any(fl)) d[fl, it] <- sample(0:MAXCAT, sum(fl), replace = TRUE)
  }
  d <- as.data.frame(d); names(d) <- paste0("q", 1:9); d
}

bounds_at <- function(results, width, items) {
  vapply(items, function(it) {
    v <- results$InfitMSQ[results$Item == it]
    as.numeric(ggdist::hdci(v, .width = width))[1:2]
  }, numeric(2))
}

flag_code <- function(msq, lo, hi) ifelse(msq > hi, 2L, ifelse(msq < lo, 1L, 0L))

data.frame(item = paste0("q", PH_MIS), location = round(PH_LOCS[PH_MIS], 2),
  latent_sd = round(PH_SD, 2), noise_p = NOISE_P)

```

	item	location	latent_sd	noise_p
q5	q5	-0.15	1.39	0.15
q8	q8	1.40	1.39	0.15

Run

```

one_rep <- function(g) {
  out <- try({
    rep <- g$rep; k <- g$k; n <- g$n
    d <- sim_noise(n, k, seed = 7e8 + 1e6 * k + 1e3 * n + rep)
    items <- names(d)
    obs <- RMitemInfit(d, output = "dataframe")
    msq <- obs$Infit_MSQ[match(items, obs$Item)]
    hd <- list(); pv <- list()
    for (j in seq_along(BS)) {
      co <- try(RMitemInfitCutoff(d, iterations = BS[j], parallel = FALSE,
        seed = rep * 100 + j, verbose = FALSE),
        silent = TRUE)
      if (inherits(co, "try-error")) next
      hd[[length(hd) + 1L]] <- do.call(rbind, lapply(WIDTHS, function(w) {
        b <- bounds_at(co$results, w, items)
        data.frame(B = BS[j], w = w, item = seq_along(items),
          code = flag_code(msq, b[1, ], b[2, ]))
      })))
    }
    fw <- try(suppressWarnings(
      RMitemInfit(d, cutoff = co, p_value = TRUE, correction = "fwer",
        alpha = ALPHA, output = "dataframe")), silent = TRUE)
    bh <- try(suppressWarnings(
      RMitemInfit(d, cutoff = co, p_value = TRUE, correction = "fdr_bh",
        alpha = ALPHA, output = "dataframe")), silent = TRUE)
    if (!inherits(fw, "try-error") && !inherits(bh, "try-error")) {
      i1 <- match(items, fw$Item); i2 <- match(items, bh$Item)
      pv[[length(pv) + 1L]] <- data.frame(
        B = BS[j], item = seq_along(items),
        padj_fwer = fw$padj_infit[i1], padj_bh = bh$padj_infit[i2])
    }
  })
}

```

```

    }
    list(hdci = do.call(rbind, hd), pval = do.call(rbind, pv))
  }, silent = TRUE)
  if (inherits(out, "try-error")) NULL else out
}

grid <- expand.grid(rep = 1:R_REPS, n = NS, k = KS)[, c("rep", "k", "n")]
grid <- grid[order(grid$n), ]
rownames(grid) <- NULL
cat("cells to run:", nrow(grid), "\n")

```

cells to run: 480

```

s <- run_resumable(RES, grid, c("rep", "k", "n"), one_rep,
  extra = list(mis = PH_MIS, locs = PH_LOCS, noise_p = NOISE_P))

bind_part <- function(obj, part, mech) {
  do.call(rbind, lapply(seq_len(nrow(obj$grid)), function(i) {
    r <- obj$res[[i]][[part]]
    if (is.null(r)) return(NULL)
    cbind(mech = mech, obj$grid[i, c("rep", "k", "n")], r)
  })))
}

mis_of <- function(k) PH_MIS[seq_len(k)]

NOI_H <- bind_part(s, "hdci", "noise")
NOI_P <- bind_part(s, "pval", "noise")

# --- comparator: rho = 0.85 phq9 cells from the poly_weak study -----
pw <- readRDS(COMPARATOR)
keep <- which(pw$grid$arm == "phq9" & abs(pw$grid$rho - 0.85) < 1e-9)
MD_H <- do.call(rbind, lapply(keep, function(i) {
  r <- pw$res[[i]]$hdci; if (is.null(r)) return(NULL)
  cbind(mech = "multidim", pw$grid[i, c("rep", "k", "n")], r) })))
MD_P <- do.call(rbind, lapply(keep, function(i) {
  r <- pw$res[[i]]$pval; if (is.null(r)) return(NULL)
  cbind(mech = "multidim", pw$grid[i, c("rep", "k", "n")], r) })))

# The poly_weak comparator also stores `p_marg`, which this study does not
# record; bind on the shared columns only.
bind_common <- function(a, b) {
  nm <- intersect(names(a), names(b))
  rbind(a[, nm, drop = FALSE], b[, nm, drop = FALSE])
}

H <- bind_common(NOI_H, MD_H) |>
  filter(n %in% NS, B %in% BS) |>
  mutate(is_misfit = mapapply(function(k, it) it %in% mis_of(k), k, item),
    flag = as.integer(code > 0), w = sprintf("%.3f", w))
P <- bind_common(NOI_P, MD_P) |>
  filter(n %in% NS, B %in% BS) |>
  mutate(is_misfit = mapapply(function(k, it) it %in% mis_of(k), k, item))

cat(nrow(s$grid), "noise cells |", round(s$elapsed_min, 1), "min | failed:",
  sum(sapply(s$res, is.null)), "\n")

```

480 noise cells | 96 min | failed: 0

```
cat("comparator cells:", length(keep), "\n")
```

```
comparator cells: 720
```

The cascade: false positives on fitting items

The quantity at issue. Both mechanisms are matched on target-item infit at $k = 1$.

```
H |>
  filter(!is_misfit, B == 2000, w == "0.999") |>
  group_by(mech, k, n) |>
  summarise(fp = round(100 * mean(flag), 1), .groups = "drop") |>
  pivot_wider(names_from = mech, values_from = fp) |>
  mutate(ratio = round(multidim / pmax(noise, 0.01), 1)) |>
  arrange(k, n)
```

k	n	multidim	noise	ratio
1	250	0.4	0.4	1.0
1	1000	1.4	0.7	2.0
2	250	0.5	1.4	0.4
2	1000	1.7	11.3	0.2

Same, for the FWER rule:

```
P |>
  filter(!is_misfit, B == 2000) |>
  mutate(f = as.integer(padj_fwer < ALPHA)) |>
  group_by(mech, k, n) |>
  summarise(fp = round(100 * mean(f, na.rm = TRUE), 1), .groups = "drop") |>
  pivot_wider(names_from = mech, values_from = fp) |>
  arrange(k, n)
```

k	n	multidim	noise
1	250	1.0	0.3
1	1000	3.2	1.9
2	250	0.7	2.5
2	1000	4.8	22.5

Direction of the false flags

If the cascade is driven by a correlated cluster, its false positives should be overwhelmingly **overfit**. Independent noise should show less of that skew.

```
H |>
  filter(!is_misfit, B == 2000, w == "0.999", code > 0) |>
  mutate(dir = ifelse(code == 1, "overfit", "underfit")) |>
  group_by(mech, k, n, dir) |>
  summarise(ct = n(), .groups = "drop") |>
  group_by(mech, k, n) |>
  mutate(pct = round(100 * ct / sum(ct), 1)) |>
  ungroup() |> select(-ct) |>
  pivot_wider(names_from = dir, values_from = pct, values_fill = 0) |>
  arrange(mech, k, n)
```

mech	k	n	overfit
multidim	1	250	100
multidim	1	1000	100

mech	k	n	overfit
multidim	2	250	100
multidim	2	1000	100
noise	1	250	100
noise	1	1000	100
noise	2	250	100
noise	2	1000	100

Detection of the genuinely misfitting items

```
H |>
  filter(is_misfit, B == 2000, w == "0.999") |>
  group_by(mech, k, n) |>
  summarise(det = round(100 * mean(flag), 1), .groups = "drop") |>
  pivot_wider(names_from = mech, values_from = det) |>
  arrange(k, n)
```

k	n	multidim	noise
1	250	46.7	50.8
1	1000	99.2	100.0
2	250	15.0	38.3
2	1000	77.9	95.4

Per item, to separate the on-target (q5) from the off-target (q8) case:

```
H |>
  filter(is_misfit, B == 2000, w == "0.999") |>
  mutate(loc = round(PH_LOCS[item], 2)) |>
  group_by(mech, k, n, item, loc) |>
  summarise(det = round(100 * mean(flag), 1), .groups = "drop") |>
  pivot_wider(names_from = mech, values_from = det) |>
  arrange(k, n, loc)
```

k	n	item	loc	multidim	noise
1	250	5	-0.15	46.7	50.8
1	1000	5	-0.15	99.2	100.0
2	250	5	-0.15	23.3	25.0
2	250	8	1.40	6.7	51.7
2	1000	5	-0.15	91.7	90.8
2	1000	8	1.40	64.2	100.0

Does FWER keep its advantage under the other mechanism?

```
bind_rows(
  H |> filter(B == 2000, w == "0.999") |>
    group_by(mech, k, n) |>
    summarise(rule = "HDCI.999", det = 100 * mean(flag[is_misfit]),
              fp = 100 * mean(flag[!is_misfit]), .groups = "drop"),
  P |> filter(B == 2000) |>
    mutate(fwer = as.integer(padj_fwer < ALPHA),
           bh = as.integer(padj_bh < ALPHA)) |>
```

```

pivot_longer(c(fwer, bh), names_to = "rule", values_to = "f") |>
group_by(mech, k, n, rule) |>
summarise(det = 100 * mean(f[is_misfit], na.rm = TRUE),
          fp = 100 * mean(f[!is_misfit], na.rm = TRUE), .groups = "drop")
) |>
mutate(across(c(det, fp), \(x) round(x, 1))) |>
arrange(mech, k, n, rule)

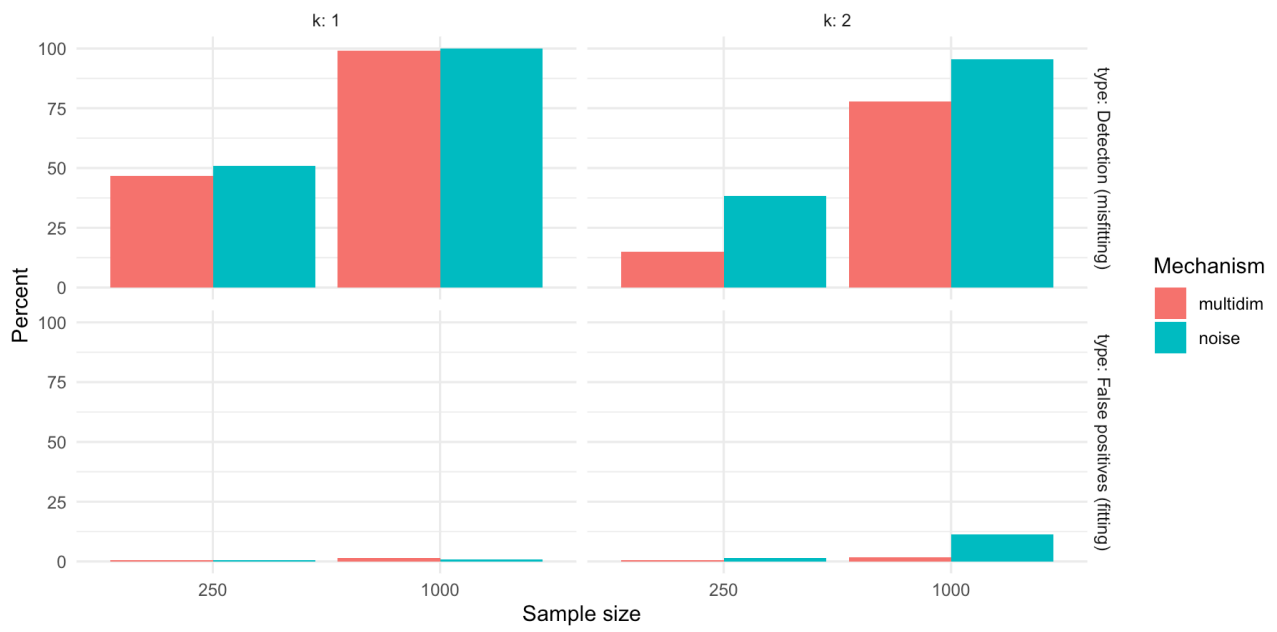
```

mech	k	n	rule	det	fp
multidim	1	250	HDCI.999	46.7	0.4
multidim	1	250	bh	62.5	1.8
multidim	1	250	fwer	62.5	1.0
multidim	1	1000	HDCI.999	99.2	1.4
multidim	1	1000	bh	100.0	5.2
multidim	1	1000	fwer	100.0	3.2
multidim	2	250	HDCI.999	15.0	0.5
multidim	2	250	bh	32.9	1.8
multidim	2	250	fwer	31.7	0.7
multidim	2	1000	HDCI.999	77.9	1.7
multidim	2	1000	bh	90.8	8.8
multidim	2	1000	fwer	86.7	4.8
noise	1	250	HDCI.999	50.8	0.4
noise	1	250	bh	66.7	0.5
noise	1	250	fwer	67.5	0.3
noise	1	1000	HDCI.999	100.0	0.7
noise	1	1000	bh	100.0	4.1
noise	1	1000	fwer	100.0	1.9
noise	2	250	HDCI.999	38.3	1.4
noise	2	250	bh	59.6	3.9
noise	2	250	fwer	55.4	2.5
noise	2	1000	HDCI.999	95.4	11.3
noise	2	1000	bh	99.2	37.5
noise	2	1000	fwer	98.3	22.5

```

H |>
filter(B == 2000, w == "0.999") |>
mutate(type = ifelse(is_misfit, "Detection (misfitting)",
                    "False positives (fitting)") |>
group_by(mech, k, n, type) |>
summarise(pct = 100 * mean(flag), .groups = "drop") |>
ggplot(aes(factor(n), pct, fill = mech)) +
geom_col(position = "dodge") +
facet_grid(type ~ k, labeller = label_both) +
labs(x = "Sample size", y = "Percent", fill = "Mechanism") +
theme_minimal(base_size = 11)

```



References

Johansson, M. (2025). Detecting Item Misfit in Rasch Models. *Educational Methods & Psychometrics*, 3(18).

Tuerlinckx, F., & De Boeck, P. (2001). The effect of ignoring item interactions on the estimated discrimination parameters in item response theory. *Psychological Methods*, 6(2), 181-195.